

Level 2 - Sorting

1. Merge Sorting

```
arr = [5, 3, 8, 1]

def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2

    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)

def merge(left, right):

    result = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):

        if left[i] <= right[j]:
            result.append(left[i])
            i += 1

        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result

print(merge_sort(arr))
```

2. Quick Sorting

```
def quick_sort(arr, low, high):
    if low < high:
        pivot_index = partition(arr, low, high)
        quick_sort(arr, low, pivot_index - 1)
        quick_sort(arr, pivot_index + 1, high)

def partition(arr, low, high):
    pivot = arr[high]
    i = low

    for j in range(low, high):
        if arr[j] < pivot:
            arr[i], arr[j] = arr[j], arr[i]
            i += 1
    arr[i], arr[high] = arr[high], arr[i]
    return i

arr = [5, 3, 8, 1, 2]
quick_sort(arr, 0, len(arr) - 1)
print(arr)
```

3. Python Sorting

1 `sort()`: `list.sort()`
original list modified and sorted arranges in Asc order.

```
nums = [5, 2, 8, 1, 3]
nums.sort()
print(nums)
Output:
[1, 2, 3, 5, 8]
```

Important:
`sort()`
↓
Original list changes

2 `sorted()`: `sorted()`
original iterable without changing new sorted list will returns.

```
nums = [5, 2, 8, 1, 3]
result = sorted(nums)
print(nums)
print(result)
```

Output:
nums → [5, 2, 8, 1, 3]
result → [1, 2, 3, 5, 8]

🔥 Difference:

`sort()`
→ modifies original list

`sorted()`
→ returns new sorted list